	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 9

Lab Title: **BST ,insertion deletion and searching techniques**

Student Name: **Muammad Waqas** **Reg. No:** **240756**

Objective:understanding and im[plementation of BST and insertion deletion and searching techniques in . BST and and real life applications

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

Department of Computer Engineering

Data Structures And Algorithms

LAB REPORT : 8

Tittle : Priority queue

Name : Muammad waqas

Roll No: 240756

Submitted To : CE . Engr Maryam

Date : 2 December 24

Objective

Binary Search Tree is a node-based binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the
- node's key.
- The right subtree of a node contains only nodes with keys greater than the
- node's key.
- The left and right subtree each must also be a binary search tree.

TASK 1

1) Implement a binary search tree by inserting nodes. Tree should be up to5 levels.
2) Search the last element in the binary search tree.
3) Delete node in BST having 1 child
4) Delete leaf node in BST
5) Delete root node in BST. (Bonus question)

====(CODE)====

```
`#include<iostream>
using namespace std;
class node{
public:
int d;
node *l,*r;
node(int x){
d=x;
l=r=NULL;}
};
class BST{
public:
node* root;
int lvl;
BST(){root=NULL;lvl=0;}
node* insert(node* t,int x,int depth){
if(depth>5){
cout<<"Tree level limit (5) reached.\n";
return t;
}
if(t==NULL){
t=new node(x);
return t;
}
if(x<t->d) t->l=insert(t->l,x,depth+1);
else t->r=insert(t->r,x,depth+1);
return t;
```

====(CODE)====

```
28  }
29  void add(int x){
30  root=insert(root,x,1);
31  }
32  void inorder(node* t){
33  if(t==NULL) return;
34  inorder(t->l);
35  cout<<t->d<<" ";
36  inorder(t->r);
37  }
38  bool searchLast(node* t,int x){
39  if(t==NULL) return false;
40  if(t->d==x) return true;
41  if(x<t->d)
42  return searchLast(t->l,x);
43  else{
44  return searchLast(t->r,x);}}
45  node* delNode(node* t,int x){
46  if(t==NULL) return NULL;
47  if(x<t->d) t->l=delNode(t->l,x);
48  else if(x>t->d) t->r=delNode(t->r,x);
49  else{
50  // Case 1: Leaf node
51  if(t->l==NULL && t->r==NULL){
52  delete t;
53  return NULL;}
54  // Case 2: One child
```

====(CODE)====

```
55  else if(t->l==NULL){
56  node* tmp=t->r;
57  delete t;
58  return tmp;}
59  else if(t->r==NULL){
60  node* tmp=t->l;
61  delete t;
62  return tmp;}
63  // Case 3: Two children
64  else{
65  node* succ=t->r;
66  while(succ->l!=NULL) succ=succ->l;
67  t->d=succ->d;
68  t->r=delNode(t->r,succ->d); }
69  }
70  return t;}
71  void del(int x){
72  root=delNode(root,x);}
73  };
74  int main(){
75  BST b;
76  int ch,x,last=0;
77  do{
78  cout<<"\n===== BST MENU =====\n";
79  cout<<"1.Insert node\n";
80  cout<<"2.Search last inserted element\n";
81  cout<<"3.Delete node with 1 child\n";
```

====(CODE)====

```
 82  cout<<"4.Delete leaf node\n";
 83  cout<<"5.Delete root node \n";
 84  cout<<"6.Display inorder\n";
 85  cout<<"7.Exit\n";
 86  cout<<"Enter choice: ";
 87  cin>>ch;
 88  if(ch==1){
 89  cout<<"Enter value: ";
 90  cin>>x;
 91  last=x;
 92  b.add(x);}
 93  else if(ch==2){
 94  cout<<"Searching last inserted "<<last<<"...\n";
 95  if(b.searchLast(b.root,last)) cout<<"Found\n";
 96  else cout<<"Not Found\n";
 97  }
 98  else if(ch==3){
 99  cout<<"Enter value (1 child): ";
100  cin>>x;
101    b.del(x);}
102    else if(ch==4){
103  cout<<"Enter leaf node value: ";
104  cin>>x;
105  b.del(x);}
106  else if(ch==5){
107  cout<<"Deleting root: "<<
108  (b.root? b.root->d:0)<<endl;
```

====(CODE)====

```
109     if(b.root) b.del(b.root->d);
110       }
111       else if(ch==6){
112     cout<<"Inorder: ";
113     b.inorder(b.root);
114     cout<<endl;
115     }
116     else if(ch!=7){
117     cout<<"Invalid\n";
118       }
119     }while(ch!=7);
120     return 0;
121     }
```

====(OUTPUT)====

MENU

C:\Users\Nextsol Pvt Ltd\Downloads\t1

```
===== BST MENU =====
1.Insert node
2.Search last inserted element
3.Delete node with 1 child
4.Delete leaf node
5.Delete root node
6.Display inorder
7.Exit
Enter choice:
```

Insertion first node as Root node

C:\Users\Nextsol Pvt Ltd\Downloads\t111

```
===== BST MENU =====
1.Insert node
2.Search last inserted element
3.Delete node with 1 child
4.Delete leaf node
5.Delete root node
6.Display inorder
7.Exit
Enter choice: 1
Enter value: 3
```

====(OUTPUT)====

Searching Function

```
===== BST MENU =====
1.Insert node
2.Search last inserted element
3.Delete node with 1 child
4.Delete leaf node
5.Delete root node
6.Display inorder
7.Exit
Enter choice: 2
Searching last inserted 7...
Found
```

Delete node with one child

```
===== BST MENU =====
1.Insert node
2.Search last inserted element
3.Delete node with 1 child
4.Delete leaf node
5.Delete root node
6.Display inorder
7.Exit
Enter choice: 3
Enter value (1 child): 8
```

Delete Root node

```
===== BST MENU =====
1.Insert node
2.Search last inserted element
3.Delete node with 1 child
4.Delete leaf node
5.Delete root node
6.Display inorder
7.Exit
Enter choice: 6
Inorder:  -4 1 2 5 6 7 9
```

Conclusion :

- A Binary Search Tree keeps data ordered so every insert places smaller values on the left side and larger values on the right side.
- Searching becomes easy because at each step you only move left or right instead of checking every node.
- Insertion is simple: follow the BST rule until you find an empty spot and place the new value there.
- Deleting a leaf node is the easiest case because you just remove it without affecting other parts of the tree.
- Deleting a node with one child means you remove the node and connect its child directly to the parent.
- Deleting a node with two children is handled by replacing it with its inorder successor so the BST property stays correct.
- Overall, BST operations help store and manage data in a structured way that keeps searching, inserting, and deleting efficient and organized.